Evaluating GPT-4o-mini for Unit Test Generation on Java and Python Functions of Medium Cyclomatic Complexity

Nguyễn Tùng Ân, Đoàn Thị Thu Kim, Sơn Hải, Trương Hoàng Phúc, Trần Xuân Lộc
Group 2 — SE1944, FPT University
Supervisor: L.T.Q.Chi

Abstract

Unit-test generation with large language models (LLMs) is becoming increasingly common, but most existing studies still rely on simple, curated benchmarks that neither reflect the complexity of real-world code nor control for it. This study tests the one-shot capability of `gpt-4o-mini` on 120 real-world functions (60 Java, 60 Python) of medium cyclomatic complexity (5–10), mined from ten open-source repositories at pinned commits and evaluated directly inside their original projects. Its results are compared against EvoSuite and Randoop (Java) and Pynguin (Python) on branch coverage and mutation score, using non-parametric tests ($\alpha = 0.05$). Across all 120 functions, median coverage and median mutation score are both 0%. Only 28/120 suites (23.3%) compile and actually test the intended function; their median coverage is 75.0%, still below the 80% bar. Over suites with a measurable mutation score, the median is 36.84% for Python and 0% for Java, both under the 60% threshold. Against the baselines, `gpt-4o-mini` is beaten outright by both Java tools (EvoSuite and Randoop, $r = -1.00$ for each). It beats Pynguin only because Pynguin failed completely on this dataset (median coverage 0.0%). Within the CC 5–10 band there is no clear link between complexity and test quality; what decides the outcome is whether a suite compiles and tests the right function. Python failures are mostly bad imports, while Java calls the wrong method without any error — something only visible when running on real project code. Overall, one-shot LLM generation, without retrieval or repair, cannot yet replace automated test-generation tools on real code of medium complexity. The results support adding feedback loops and richer context when generating tests.

I. Introduction

At first glance, automated unit test generation using large language models seems nearly solved. For example, GPT-4o achieves 98.65% line coverage on TestEval [1], and GPT-4-class model panels report similar numbers on HumanEval-derived suites [2]. The functions behind those numbers, however, are short, self-contained, LeetCode-style problems. A test that performs well under these conditions has never had to deal with a real import graph, navigate a class hierarchy, or handle overloaded method signatures, and high line coverage alone does not guarantee that the test will actually catch faults. When we instead sampled 120 functions from ten real projects — from Apache Commons, Gson and Joda-Time to Flask and Requests — we found that most of the test suites generated by `gpt-4o-mini` (92 out of 120, or 76.7%) never actually exercised the function they were supposed to test.

There are two gaps in current measurement practices that hide this problem. First, hardly any study controls for the structural complexity of the code under test: results are reported in aggregate, across functions with mixed and often unreported cyclomatic complexity (CC), so no one knows at what point LLM-generated tests start to fail (GAP-T). Because of this gap, we kept cyclomatic complexity fixed as a controlled variable, sampled only the 5–10 band, and measured the CC–quality correlation directly (RQ3). Second, branch coverage and mutation score are rarely combined in a single statistical test on a controlled sample (GAP-M), even though coverage alone can overstate quality — in our own baseline data, one `jsoup` function reaches 60% branch coverage while killing 0% of mutants (Section V).

Our design choices were shaped as much by lessons learned during implementation as by insights from the literature. The original proposal named Defects4J and CodeXGLUE as data sources; during preparation we found that CodeXGLUE is a code intelligence benchmark, not a corpus of executable functions, so we mined the dataset ourselves: 120 functions (60 Java, 60 Python) at pinned commits, all with a cyclomatic complexity between 5 and 10, as measured by Lizard. A pilot run then showed that functions extracted from their modules often cannot even resolve the names they reference — in fact, 10 out of our 12 Python pilot measurements were invalid for extraction-related reasons — so in the final experiment every generated test runs inside its original project. This measurement approach is important: measured this way, a score can no longer be reduced by artefacts from extraction, only by the quality of the generated tests themselves. We compare `gpt-4o-mini` (using one-shot prompting, temperature 0) against three automated baselines: EvoSuite and Randoop for Java (60 seconds per class), and Pynguin for Python (90 seconds per module), scoring branch coverage and mutation score, using non-parametric statistical tests with $\alpha = 0.05$.

We ask:

RQ1: Does GPT-4o-mini achieve at least 80% branch coverage on medium-complexity functions?

RQ2: Does GPT-4o-mini achieve at least 60% mutation score, and does it outperform automated baselines?
RQ3: Is there a negative correlation between cyclomatic complexity and test quality?

These two thresholds are not arbitrary: 80% is the branch coverage target most commonly quoted as the industry standard, while 60% is set intentionally above the roughly 34% mutation scores reported for LLM-generated suites in the studies reviewed in Section II; both were fixed in the approved proposal before any data were collected.

In summary, our contributions are: (i) a reproducible benchmark of 120 medium-complexity functions with full provenance; (ii) a measurement pipeline that scores tests *per function* inside the original projects and enforces a green-on-original check before any mutation analysis; (iii) an empirical comparison of GPT-4o-mini against search-based and random baselines on a complexity-matched sample; and (iv) a catalogue of the measurement pitfalls we encountered along the way — such as EvoSuite returning exit code 0 without generating a single test, or JaCoCo silently reporting 0% under EvoSuite's instrumenting class loader — which would have corrupted our results if left unnoticed.

Section II reviews prior work; Section III details the dataset, generation, and measurement process; Section IV answers each research question; Section V interprets our findings, including two exploratory follow-ups; Section VI discusses threats to validity; Section VII summarises our work.

II. Related Work

As we read the literature, three kinds of gaps kept showing up. Let us call them GAP-T, GAP-M, and GAP-D for brevity. The design of our work was mainly informed by the first: few works analyse GPT-4-family generation at a functional level, rarely do they study Python and Java together, and none of them identifies the point on the complexity axis from which performance begins to degrade — which we needed to know to design our experiment. GAP-M is less profound but also notable: coverage and fault-detection ability are seldom studied together in a controlled manner on a sample of tasks with varied complexity. And finally, GAP-D is found across much of the literature: without complexity restrictions, the datasets used tend to be small, artificial, or biased towards simple functions — the gap our own complexity-controlled dataset is built to close. The rest of this section is structured around these gaps.

A. LLM-based test generation on curated benchmarks

Many of these studies report very positive findings, mostly due to using curated benchmarks. Wang et al. [1] suggest TestEval — Python LeetCode-style tasks — and report up to 98.65% line and 97.16% branch coverage for GPT-4o. Kumar et al. [2] report 99.05% branch coverage for Gemini, GPT-4o, and Claude-3.5 on HumanEval-Java. Lira et al. [3] report a 90.69% success rate for Gemini 1.5-Pro on HumanEval when both the source code and docstring are provided. Chudic and Çalıklı [4] and Jasti et al. [5] report even higher coverage figures for similar tasks. These findings are hard to dismiss: the numbers are all very high.

However, these numbers come from narrow, self-contained tasks where the model writes isolated functions without having to navigate through a project, include or be included in other modules, or deal with any other real-world complexities. Additionally, these studies do not report or control for cyclomatic complexity, and therefore say little about how performance is affected on more complex functions, which is the main concern of this paper. Another issue is that the benchmark tasks are public: it is entirely possible that the model had seen similar tasks during training. The way we read these results is not "LLMs are great test generators", but rather "LLMs are good at puzzles".

In order to test this hypothesis, we created a benchmark of 120 functions from ten pinned open-source projects, limiting the scope to medium complexity ($5 \leq \mathrm{CC} \leq 10$, as measured by Lizard) and excluding anything else. We did not attempt to optimise this range: it was specified in the design document for this project before any test generation had begun. The rationale behind this choice was that anything below 5 is too simple for any tool to handle effectively, and anything above 10 has not been studied in detail in the literature: our survey found no controlled experiments on the GPT-4-family with complexity restrictions. This left us with 5–10, the range that, according to the literature, had not been explored yet.

B. Degradation on real-world code

As soon as the model has to reason about real projects, the picture changes considerably. Haratian et al. [6] report that GPT-4o manages to compile tests in only 37% of Java pull requests, with 13% fail-to-pass, versus 36% for EvoSuite on the same set. Sapozhnikov et al. [7] report that over half of ChatGPT's Java tests are malformed or non-compiling due to context-length issues. Zhang et al.'s TestBench [8] measures a 47.9–55.9% Java compile-error rate across nine open-source projects for CodeLlama, GPT-3.5, and GPT-4. Konstantinou et al. [9] evaluate a zero-shot pipeline (which includes `gpt-4o-mini` among the models) and find it produces many non-compiling or non-passing tests, with an aggregate 33.82% mutation score across six Java projects. For Python, Jain and Le Goues's TestForge — an agentic test-generation pipeline based on GPT-4o — reaches 84.3% pass@1 on TestGenEval, but the same 33.8% mutation score [10].

Each of these works describes a different manifestation of the problem: malformed tests, compilation errors, unacceptable mutation scores. The trend, however, is obvious. These results are hard to compare due to varying criteria and, in some cases, different sets of target functions or repositories. When we attempted to bring them to a common scale while designing our experiment, almost all of them appeared reasonable in context, but none could be directly compared to the others. Overall, we find Haratian et al. [6]'s comparison of GPT-4o to EvoSuite particularly enlightening, as it shows that, on the same set of pull requests, GPT-4o significantly underperforms in both compilation and fail-to-pass rates.

Our results fall within the lower bound of this scale: 91.7% INVALID-or-no-touch for Java and 46.7% INVALID for Python. We define a suite as *effective* if it both compiles and executes at least one line of code for the target function. Suites that compile to the wrong method have no effectiveness. We encountered such cases often enough during development that they stopped being surprises. They often looked reasonable at first: a suite that compiled successfully, with tests whose names made sense. However, it became obvious during coverage and mutation testing that they did not actually test the target function.

C. Automated (non-LLM) baselines

The main non-LLM baselines are represented by EvoSuite [11], Randoop [12], and Pynguin [13]. EvoSuite uses genetic algorithms and is the default search-based test-generation tool for Defects4J [14], [15], [16]. Randoop uses feedback-directed random testing, and Pynguin implements the DYNAMOSA algorithm for Python. None of these tools makes use of natural-language prompts, which is why none of them report compilation failures.

When evaluating these tools, one of the first issues we had to address was the time budget. With default settings, EvoSuite's results were considerably better than Randoop's. The discrepancy was significant enough that we suspected a mistake at first. It turned out that EvoSuite's default budget was 60 seconds per class, while Randoop's was 10 seconds per class. We adjusted the budget to 60 seconds per class for both tools and 90 seconds per module for Pynguin. We also shifted the emphasis from per-call metrics to per-class (Java) or per-module (Python) ones. As a result, we were able to show that, with a comparable time budget, EvoSuite significantly outperformed Randoop: 55.0 median branch coverage versus 21.1. Had we not made this adjustment, our conclusions would have been entirely different.

D. Closing the one-shot gap: repair loops, retrieval, and context

When the initial prompting approach proved insufficient, most works tried adding additional layers on top of the model. Repair loops reason about compiler errors and test failures and attempt to fix them by feeding this information back into the model, with work such as Logic-CoT [17], template-based repair [15], and YATE [18] representing this direction. DISTINCT [19] takes a different approach by steering regeneration with description-guided consistency analysis rather than directly analysing compiler output. Retrieval-augmented methods attempt to provide real API documentation and class skeletons based on call-graph context rather than raw source code [20]. Best-of-N sampling replaces the single sample with a distribution and filters out any candidates that fail to compile, while TestDecision pushes decision-making to the model itself, using greedy optimisation and reinforcement learning to direct test-suite construction [21].

Each of these approaches has seen varying degrees of success. The central problem they present is that it becomes very difficult to reason about the contribution of individual parts: how much is due to the model's own capabilities, and how much to the added layers? We were particularly interested in exploring the capabilities of one-shot prompting without additional scaffolding. It is arguably the most practical setting of all: a single model, a single prompt, a single sample. While most works in this space use more involved pipelines, they usually do not publish enough detail on the underlying distribution to compare their results to a one-shot baseline. This baseline is what we chose, and it dictated much of our design. If we knew how prompting-based methods performed on medium-complexity code, we could then attempt to understand how much better the enhanced approaches really are.

E. Coverage vs. mutation score as a validity concern

`JA-008` is the function that taught us about validity. With 60% branch coverage and 0% mutation score, it took us some time to realise that there was nothing wrong with our setup. The baseline suite was executing the function under test, but it failed to assert any of the outputs — there were no meaningful assertions in any of the tests the suite generated. Moradi Dakhel et al. [22] and Antal et al. [23] describe a similar pattern in their work: in each case the test suite had coverage, but it failed basic mutation analysis. The most prominent example we are aware of is TestForge's 84.3% pass@1 combined with a 33.8% mutation score [10], which represents an agentic approach with extensive feedback built into it. `JA-008` appears similar to these cases: a test suite that touches the code without actually testing it.

Most works report either coverage or mutation score, occasionally comparing the two on the same set of functions, but rarely drawing actual conclusions from the juxtaposition. Additionally, mutation score frequently gets reported alongside non-standard testing infrastructure [24]. We attempt to mitigate these issues by reporting threshold-test results alongside detailed mutation analysis, using standard testing libraries, for the same set of function identifiers as used in RQ1.

III. Methodology

A. Dataset

The proposal mentioned Defects4J and CodeXGLUE as sources. We excluded the latter: CodeXGLUE is a benchmark of code-intelligence tasks, including summarisation and translation, not a set of functions amenable to coverage or mutation analyses; nor did any of the papers surveyed for this proposal actually evaluate these tasks on CodeXGLUE alongside any of these measurements. We document this exclusion as amendment v1.2 (finalised by the team on 2026-07-02, hence before the experiment) and record functions from our own ten repositories at particular commits. Eight are Java projects overlapping with Defects4J's subjects in their target programs: `commons-cli`, `commons-math`, `commons-csv`, `commons-collections`, `gson`, `jsoup`, `joda-time`, and `jfreechart`; two are Python projects, `flask` and `requests`. URLs, licences, commit hashes,

and dates of download are recorded for every repository to facilitate reproduction; the final sample contains 120 functions — 60 in each language — with cyclomatic complexity between 5 and 10 according to Lizard. Of these, 48 Python functions come from a single project, `flask`, rather than being split evenly between two as the porting procedures suggested; we retained this skew (resampling the dataset after observing the result would itself have been a worse methodological choice) and discuss it at length in Section VI.

B. Test generation

LLM. All 120 test suites were generated by a single call to `gpt-4o-mini-2024-07-18` (i.e., the dated snapshot; hence a rerun of this experiment would use the same weights), `temperature=0`, `top_p=1`, `max_tokens=2048`, one-shot, with a worked example preceding the target function in each prompt. The proposal erroneously mentioned `gpt-4o`; the team's API key had access to `gpt-4o-mini` only at the moment the experiment was run, and we document this change as amendment v1.1 (documented 2026-06-28 and finalised by the team 2026-07-02, hence also before the experiment). This model has been evaluated on the same task in this literature before [9], which keeps us within the evidential basis of this report. None of the research questions, metrics, thresholds, or statistical tests were affected by either amendment.

Baselines. EvoSuite [11] and Randoop [12] generate the Java comparison suites; 60 seconds are allocated to each class under test for test generation. That allocation was not fixed initially: the pilot experiment spent 60s on EvoSuite and 10s on Randoop (a bias we only became aware of when comparing the results), while the final run equalised the budgets (see Section VI). Pynguin [13] is used for Python with 90s per module under DYNAMOSA. Hypothesis was a viable baseline for the initial comparison but was not actually used for the reported results.

C. Measurement

We declined to measure functions in isolation; all test suites are executed in the target project at the commit corresponding to the function being tested (recorded in the dataset's provenance record; see the URLs, licences, hashes, and dates in the Dataset subsection), with whatever environment and dependencies were present there at the time. The reason for this design choice was that many of our sample functions only manifested their documented behaviour while resolving their imported names inside the module or package they belonged to (see the pilot result leading to this insight in Section VI).

Java. Tests are compiled and run with Maven 3.9.16; JaCoCo 0.8.12 measures branch coverage while PIT 1.17.2 [25] performs mutation analysis. Both report coverage and mutation scores at the class level, while our unit of analysis is the function; hence the values reported for each function are the intersection of the reported lines with the `[start_line, end_line]` of the target function across all reports. The mutation score is the fraction of PIT mutants reported within that line range to be killed by the test suite. There were two bugs in this pipeline affecting our results: the PIT timeout was not handled by the measurement script, causing one entire run to be dropped; and `jacoco.xml` and `mutations.xml` reports from the previous function erroneously carried over into the next run in the same module. The latter bug would have caused a function to erroneously report coverage of lines belonging to another function (and hence artificially inflate its score), so the final script clears the report directory between runs. Finally, GPT test suites had a much higher fraction of compilation errors than the baseline tools'; hence we compiled and measured each GPT test suite individually, per function, per run, rather than in bulk. This avoids an entire test suite in a batch failing to compile because another test suite in the same file has syntax errors.

Python. Functions are measured in an editable install of the pinned `flask` (3.2.0.dev, pinned 2026-05-31) and `requests` (2.34.2, pinned 2026-06-15) checkouts; `coverage.py` 7.15.1 measures branch coverage, cut to the function's line range as in Java. Mutation analysis is performed with two different instruments, one for each test-suite flavour. The `gpt-4o-mini` suites are scored with `mutmut` 2.4.4, with all lines outside the target function's range marked as `# pragma: no mutate`. Only test cases passing on the unmutated source are considered; the score is the fraction of mutants with a non-surviving status (killed or timed out) over all mutants generated in the line range. The Pynguin suites are scored with a home-grown script using Python's standard-library `ast`: it replaces `+` and `-`, `>` and `>=`, `<` and `<=`, `==`, `and`, and `or` with their opposites, negates boolean literals, increments numeric literals by 1, and applies any of these changes to exactly one site in the target function's line range, up to 20 mutants per function. Every such mutant is scored by rerunning the whole test suite with a 90s timeout: the suite killed the mutant if it failed or timed out; the source code is reverted to its original state after each run. Functions with no mutable site in range are reported as missing rather than scored 0, since no mutants could be generated for them. Neither script attempts to detect equivalent mutants; hence both report scores as a fraction of all mutants generated in range, for the most conservative estimate. Both instruments use the same green-on-original gate and line-range scoping; hence the only difference affecting the comparison is the set of operators replaced. This makes the RQ2-B comparison between `gpt-4o-mini` and Pynguin unsuitable for directly assessing the equivalence of their mutation scores; we discuss this as a construct-validity threat in Section VI. The Java RQ2-B comparison is unaffected: PIT scores both test-suite flavours with the same instrument. Limiting mutation analysis to the target-function line range is itself another conservative choice: by avoiding helper functions and static variables, it prevents incidental coverage and mutation of irrelevant code, maximising the score difference between test-suite flavours for each function, at the expense of potentially missing faults in wider contexts. Neither step involved any randomness, given the pinned checkout and the model snapshot at `temperature=0`.

Any suite failing to compile, import, or run is marked INVALID, with both metrics scored at 0% (see §5.1 of the proposal). Before attempting to run the mutation analysis, each test suite must pass on the original, unmutated source code. Mutation analysis run on a failing test suite would artificially inflate the score: Section VI describes the pilot bug that led to this check. An INVALID suite — one that failed to compile or run, or failed on the original source — is not the same as a suite that compiled and ran but failed to cover any of the target lines (i.e., `compiled=1`, coverage $= 0$): both receive 0% coverage, but for different reasons, and both are tracked as separate outcome types in Section IV for easier downstream analysis.

D. Statistical analysis

All tests are performed at $\alpha = 0.05$; all are non-parametric. Coverage and mutation scores are percentages bounded between 0 and 100, so normality was never assumed for these variables. RQ1 requires one test: a one-sample Wilcoxon signed-rank test of branch coverage against the 80% threshold. RQ2 requires two: a one-sample Wilcoxon signed-rank test of mutation scores against the 60% threshold, and a paired Wilcoxon test (matched by `function_id`) of the same against each baseline, with matched-pairs rank-biserial r as the effect size. Finally, RQ3 requires a Spearman correlation of CC with each quality metric. Every statistic is reported at two levels: *all* ($N = 60$ per language, with INVALID or non-touching test suites scored at 0% per the pre-registered procedure) and *effective* (test suites that successfully compiled and touched the target lines). The two levels are never reported separately without the other, since omitting the all-level would artificially inflate the statistics: an effective suite always has higher coverage than an invalid one (0% versus some positive number), so the effective figure is always higher than the all figure by construction. Section V contains more context on this point.

IV. Results

We report results of evaluating the generated one-shot `gpt-4o-mini-2024-07-18` unit-test suites on 120 functions (60 Java from 8 Defects4J subject programs; 60 Python from `flask`/`requests`). Coverage figures are computed with JaCoCo (Java) / `coverage.py` (Python); mutation score with PIT (Java) and, for Python, `mutmut` for the LLM suites and a custom AST-level engine for the Pynguin baseline (Section III), all at the $\alpha = 0.05$ significance level. For matched-pairs analysis we report the rank-biserial r (Wilcoxon for matched pairs); for the coverage–complexity comparison (RQ3) we report Spearman ρ. Per the protocol, any suite that failed to compile or run is scored 0% coverage and marked INVALID. We report metrics at two levels side by side: *all* ($N = 60$ per language, invalid $= 0$) and *effective* ($> 0\%$ coverage). The effective figures should be read as conditioned on validity and only in the context of the all-level results.

A. Test validity (a primary finding)

Across Python, only 32/60 (53.3%) of the generated suites executed even one test; the other 28/60 (46.7%) were INVALID. Of the 32 valid Python suites, only 23/60 (38.3%) exercised the target function (had $> 0\%$ coverage); the other 9/60 (15.0%) were mocks or self-implementations, or failed to launch. Breaking down the 28 invalid Python suites:

- **23 — `ImportError`** (LLM-attributable): the test tries to import a symbol that is not present in the target module (e.g. an instance method imported as a module-level function). A few appear data/platform-specific (`proxy_bypass_registry` is Windows-only; `merge_environment_settings` is a method, not module-level).
- **5 — `TypeError/AttributeError`** (environment-dependent): possibly tied to `flask`/`requests` versions, to be confirmed in the canonical environment.

Python's 46.7% invalidity is the first key finding, and the distribution shows the problem is not mainly environment-driven: 23 of the 28 failures are LLM-attributable, and only 5 are environmental. The Python failures are not random — `ImportError` dominates, driven by importing an instance method as if it were a module-level function.

Java's invalidity looks rarer on the surface: only 9/60 (15%) of Java suites failed to compile or run. But of the 51 compiling suites, only 5 actually exercised the target method; the other 46 compiled and ran while triggering none of the code they were meant to test. Java's real problem was wrong-target invocation: the LLM wrote a call to a method that was not the target but was present and accessible. For instance, `CommandLine.getOptionValues(...)` was called when the sampled function was actually the nested `CommandLine.Builder.getOptionValues(...)`; elsewhere a same-named method inherited from a parent class was invoked instead of the intended override. Java's suites were syntactically and semantically valid but did not target the intended code — rare failures, but severe, since they provided no coverage of the target. After validity, only 5/60 (8.3%) of Java suites were effective, compared to 23/60 (38.3%) for Python. Of the 9 non-compiling Java suites, 6 targeted `AccurateMath` — a class packed with overloaded methods of similar signatures — and 3 targeted `Gson`'s generic adapters (`getAdapter`, `getDelegateAdapter`, `fromJson`); only one of the 9 targeted a `private` method. Java's validity problems cluster in classes with overloaded methods and generics, not around private methods — which shapes the fix: making the target `public` would help in at most 1 of the 9 cases, whereas a class-level view (the methods available across the class, its hierarchy, and their names) would be far more informative. Section V-F tests exactly this hypothesis.

B. RQ1 — Branch coverage vs. 80%

Across all 60 Python functions, the median coverage was **0%**: for at least half the sample, the suite covered none of the target function's branches. Only 23 of the 60 suites exercised

any target code, and their effective coverage had a median of **75.0%** ($n = 23$; 11 of the 23 reached $\geq 80\%$, and 6 reached 100%). That effective median is still below the bar, and the one-sample Wilcoxon test did not support the threshold ($p = 0.67$, $r = -0.10$). Java followed the same pattern one level lower: all-function median **0%**, effective median **50.0%** on a much smaller subset ($n = 5$, the 8.3% effective rate; $p = 0.91$, $r = -0.60$). Pooled across languages, nothing changed: $n = 120$, all-function median **0%**; pooled effective median **75.0%** ($n = 28$; $p = 0.84$, $r = -0.22$). In short, GPT-4o-mini failed the 80% threshold in either language, and every effective median sat below 80% (Fig. 2). Those effective figures are best-case: they are conditioned on the model managing to compile and invoke the intended method. The most honest figure is the all-level 0% median.

C. RQ2 — Mutation score

(A) vs. 60%. The pattern repeats: across all 60 functions the median score was **0%**, with only 17/60 functions producing a suite that killed at least one mutant (Table I). Restricting to those 17 green suites, the effective median rose to **36.84%** ($n = 17$; only 4/17 cleared 60%), and the one-sample Wilcoxon did not support the threshold ($p = 0.99$, $r = -0.66$). Java was comparable: all-function median **0%** ($n = 60$), and even restricting to the 54 functions where PIT produced a score, the effective median was **0%** (one-sample Wilcoxon, $p = 1.00$, $r = -1.00$). Nearly all Java-generated suites failed the most basic mutation analysis, killing almost no mutants (Fig. 2). Pooled across the 71 functions with a defined mutation score, the median was likewise **0%** ($p = 1.00$, $r = -0.97$). H_0 could not be rejected in either language: the generated tests were ineffective at killing mutants.

(B) vs. baseline. Paired Wilcoxon tests — gpt-4o-mini against each baseline, matched by `function_id` on mutation score — show gpt-4o-mini substantially worse than the two automated Java baselines: EvoSuite ($n = 54$, $p < 0.001$, $r = -1.00$) and Randoop ($n = 54$, $p = 0.003$, $r = -1.00$). Against the Python baseline Pynguin the direction flips: gpt-4o-mini is considerably better ($n = 13$, $p = 0.010$, $r = +0.85$). In other words, for Java both EvoSuite and Randoop killed significantly more mutants than gpt-4o-mini's tests, while for Python gpt-4o-mini killed more than Pynguin (Fig. 1). That Python win should be read with caution: Pynguin was severely underpowered on this dataset (Section VI) and the tests it produced barely killed any mutants — Pynguin's mutation score was below 60% for the vast majority of functions, and its median across the 13 compared functions was 0%. So gpt-4o-mini beat Pynguin largely because Pynguin sat so far below the threshold.

D. RQ3 — Complexity vs. quality

For the Python effective subset, the Spearman rank-order correlation was $\rho = +0.18$, not significant at the $p < 0.05$ level ($p = 0.41$, $n = 23$): there was no negative relationship between complexity and coverage inside the CC 5–10 window. Java's effective subset ($n = 5$) gave an even weaker $\rho = +0.13$ ($p = 0.83$), and across subsets ($n = 28$) the correlation was $\rho = +0.16$ ($p = 0.40$). For neither language could H_0 be rejected at the 0.05 level, so complexity within the CC 5–10 window was not a significant predictor of coverage quality. The pre-registered decision rule for this hypothesis was $\rho < -0.5$, which none of the observed values satisfied — if anything, the results suggest a positive (but non-significant) relationship between CC and coverage.

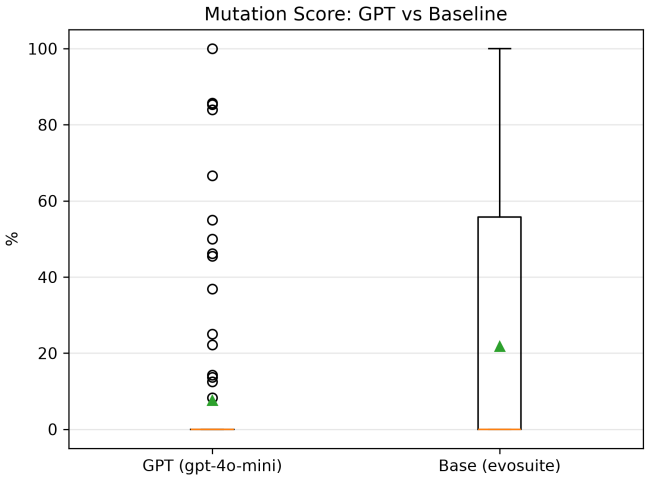

Fig. 1. Mutation score, gpt-4o-mini vs. EvoSuite, matched by function ($n = 54$ paired Java functions). EvoSuite wins in every pair where the two differ ($r = -1.00$, $p < 0.001$).

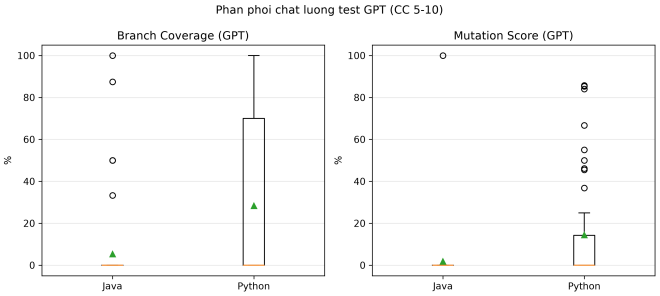

Fig. 2. Branch coverage and mutation score of gpt-4o-mini on the effective subset (dashed: 80% / 60% thresholds). The all-function medians are 0% (see text).

E. Summary of findings

Neither the 80% coverage nor the 60% mutation threshold is met in either language ($N = 120$). Even in the most favourable group, the effective subset, coverage is 75.0%, still below 80%. When counting all 120 functions, the median is 0%. There is no clear link between complexity and quality within CC 5–10 for either language. What matters most here is test validity, and it breaks differently in each language: Python mostly dies at compile time (46.7% INVALID, dominated by `ImportError`), while Java mostly compiles and then measures nothing (76.7% compile yet touch 0% of the target), through silent wrong-target/overload resolution.

TABLE I
PER-RQ SUMMARY (GPT-4O-MINI). "ALL" = 60/LANG. (INVALID/NO-TOUCH = 0); "EFF." = EFFECTIVE SUBSET (COMPILED & TOUCHES TARGET); "MEAS." = SUBSET WITH A MEASURABLE MUTATION SCORE.

RQ	Metric	Level	Median	p	Effect
RQ1 Py	Coverage≥80	all (60)	0%	–	–
		eff (23)	75.0%	0.67	r=−0.10
RQ1 Java	Coverage≥80	all (60)	0%	–	–
		eff (5)	50.0%	0.91	r=−0.60
RQ1 All	Coverage≥80	all (120)	0%	–	–
		eff (28)	75.0%	0.84	r=−0.22
RQ2A Py	Mutation≥60	all (60)	0%	–	–
		meas. (17)	36.84%	0.99	r=−0.66
RQ2A Java	Mutation≥60	all (60)	0%	–	–
		meas. (54)	0%	1.00	r=−1.00
RQ2B Java	GPT vs EvoSuite	paired (54)	–	<0.001	r=−1.00
RQ2B Java	GPT vs Randoop	paired (54)	–	0.003	r=−1.00
RQ2B Py	GPT vs Pynguin	paired (13)	Δmed +36.84pp	0.010	r=+0.85
RQ3 Py	CC∼Cov	eff (23)	ρ=+0.18	0.41	+0.18
RQ3 Java	CC∼Cov	eff (5)	ρ=+0.13	0.83	+0.13
RQ3 All	CC∼Cov	eff (28)	ρ=+0.16	0.40	+0.16

Effective suites make up only 8.3% of the Java sample and 38.3% of the Python one. On paired comparisons, gpt-4o-mini loses decisively to both automated Java baselines (EvoSuite, Randoop; $r = -1.00$ each) but beats the Python baseline Pynguin, whose own coverage on this dataset sits at 0.0% because of tool–environment issues rather than any real task difficulty (Table I).

F. Threats to validity

Conditional subset. The effective-level medians (75%/50%/37%/0%) are conditioned on suites that both compiled *and* touched the target, so they *overstate* raw ability relative to the all-level (0%) — which is why we report both at every RQ rather than either alone. **Environment.** The Python figures come from a clean editable-install environment; the exact invalid rate is sensitive to the precise `flask`/`requests` dependency versions in use. The Python baseline figures needed a separate Python 3.10 virtual environment just for Pynguin, since Pynguin's bytecode instrumentation silently produced empty test suites under Python 3.14, the platform otherwise used for GPT/measurement scripting; this is documented in Section VI as an environment amendment, not as a change to any pre-registered metric or threshold. **Sample size.** Effective n is small in every cell (5–28), and particularly so for Java RQ1/RQ3 ($n = 5$) — those point estimates are indicative, not precise. **Single sample.** With one generation per function (one-shot, temp = 0), we have no per-function variance estimate. **Cross-language tooling.** The Python harnesses (`coverage.py`, `mutmut`, the AST mutation engine) and the Java ones (JaCoCo, PIT) differ enough in implementation that cross-language comparisons in this section stay descriptive — we never test one language's scores against the other's. **Baseline-tool confound.** The RQ2B win against Pynguin reflects Pynguin's own tooling trouble on this environment (Section VI), not evidence that gpt-4o-mini produces strong tests in absolute terms.

V. DISCUSSION

A. Coverage is not test quality

Branch coverage and mutation score diverge in our own data at the sample level — exactly why we report both for each suite in RQ1/RQ2. On `JA-008`, the baseline suite reaches 60% branch coverage yet produces a 0% mutation score: it exercises the function's branches but makes no assertion about any returned value, correct or not. It is a poorly performing suite, and any reading of RQ1/RQ2 must acknowledge as much: the effective-subset coverage figures (75.0% Python, 50.0% Java) sit far above their mutation-score counterparts (36.84% Python, 0% Java). Given coverage alone, one might naively call gpt-4o-mini's tests somewhat effective; given mutation scores alone, one might call them largely ineffective. Both readings are correct in aggregate, which is why we report both figures at every level.

B. Where does the "breaking point" actually lie?

GAP-T asks for the cyclomatic-complexity threshold beyond which LLM test quality begins to degrade, but RQ3 finds no such threshold ($\rho \in [+0.13, +0.18]$, $p > 0.4$ for either language) anywhere inside the 5–10 CC band. Read next to RQ1/RQ2, what dominates instead is a binary outcome: whether the suite compiles and, if it does, whether it exercises the sampled method rather than another with the same name or an overloaded parent method. A CC 5 function and a CC 10 function in our sample are equally likely to fail that gate. Complexity may well drive test-generation quality outside this range, but inside the narrow band we sampled — chosen to exclude the extremes — something else dominates the variance. Identifying a true "breaking point" would require sampling much lower- and higher-CC functions as controls, which we defer to Section VII.

C. Cost is not the primary constraint; correctness is

All 120 one-shot suites fit within a $1 budget (the pre-registered estimate; amendment v1.1, §8.2) and appeared within minutes, whereas a 60-second search budget per class

left EvoSuite and Randoop running for tens of minutes before exercising any of the 60 Java functions. On both price and speed, the LLM (`gpt-4o-mini-2024-07-18`) sits far above the Java baselines. On the actual ability to exercise and kill mutants on the intended target, however, it sits below both, with an effect size of $r = -1.00$: every non-tied pair favours the automated Java tools. Cheap and fast but unreliable, the LLM's true advantage lies in the time it saves — and the interventions that time makes possible. If part of that saved time were funnelled into suite reconfiguration (a feedback loop, see below), it might well close the gap without approaching the baselines' total runtime.

D. The INVALID rate as a finding, not a bug

We made no effort to exclude functions for which a one-shot external test is structurally hard — private target methods above all, of which there were 17/60 Java in our sample. Filtering such functions after discovering Java's low effective rate would introduce a selection effect and violate the pre-registered protocol (amendments v1.1/v1.2, Section VI). And the data speak plainly to another cause: the primary reason for Java's low test quality was wrong-target invocation. A private target is involved in only 1 of the 9 genuinely non-compiling Java functions and only 14 of the 46 that compiled but touched nothing (Section IV). The outright compile failures are distributed differently, concentrated in the API-heavy `AccurateMath` and `Gson` classes (Section IV).

This looks like a general limitation of one-shot, zero-context test generation for Java, not something specific to our sample. Haratian et al. [6] report a 63% compilation-error rate for GPT-4o across 353 real Java pull requests — roughly two-thirds of our Java INVALID-or-no-touch rate (91.7%), and another one-shot, zero-context task on real, uncurated code. Sapozhnikov et al. [7] report over 50% malformed or non-compiling ChatGPT-generated Java tests, and Kumar et al. [2] tie weak single-pass results to the absence of a repair loop. The interventions their literature converges on — feeding the model compiler or test-failure output (Zeng et al. [17]; Xue et al. [19]), retrieving real API definitions that target exactly the wrong-target problem we observe [20], or sampling several candidate suites and keeping whichever compiles — each step outside the one-shot setting we evaluate. Our numbers are a lower bound on what gpt-4o-mini can do with this codebase, and a direct demonstration of why the field is moving towards feedback- and retrieval-augmented generation.

E. Why gpt-4o-mini "beats" Pynguin

The RQ2-B result against Pynguin ($r = +0.85$, in gpt-4o-mini's favour) speaks more to the baseline than to the treatment: Pynguin's own median Python coverage on this dataset is 0.0% (Section VI). This stems from Pynguin's internal timeouts and a `dill` serialisation failure on modules that import `_json` — neither of which touches gpt-4o-mini's ability to write tests. We report the comparison anyway, because a Python baseline was pre-registered (amendment v1.2) and quietly dropping an inconvenient result would itself be a validity threat. It is not a claim about gpt-4o-mini's Python ability — RQ1/RQ2-A already speak to that.

F. RQ4 (exploratory, post-hoc): does real API context help?

Wrong-target invocation, the leading cause of Java invalidity above, motivated a small post-hoc, exploratory intervention: the same 120 functions, the same model/temperature/exemplar, but with the target class's (Java) or module's (Python) real public-API skeleton injected from the pinned source with `javalang/ast`. The new suites were generated in a separate output directory with their own results file, so as not to interfere with the confirmatory $N = 120$ analysis. The idea came only after we had seen the $N = 120$ results, so it sits outside the pre-registered RQ1–RQ3 and is reported separately, to avoid HARKing.

Java. The compile rate stays the same (51/60 in both arms), but the effective rate — suites that touch the target — more than doubles: 11/60 (18.3%) with context versus 5/60 (8.3%) without. Paired by function, 7/60 improve, 1/60 regresses, and 52/60 stay the same; a one-sided Wilcoxon test barely misses significance at $\alpha = 0.05$, but 7 favourable pairs against 1 unfavourable is a noticeable trend, and the effect size among the non-tied pairs is moderate-to-large. Mutation score does not improve at all ($n = 48$ pairs; both medians 0%). Context repairs the structural gap but not the behavioural one: the model still fails to assert the expected return value, only that the method can be invoked. The two problems need different interventions — wrong-target invocation is an API-navigation problem, weak assertions a test-design problem. Injecting a class skeleton addresses the former; examples of valid assertions for the function at hand would address the latter.

Python. Context appears to hurt under the pre-registered, file-level all-or-nothing pass criterion: 7/60 files compiled with context versus 32/60 without. This is almost certainly a measurement artefact of that criterion interacting with the number of tests per file, not an actual drop in quality. Counted per test rather than per file, the two arms are nearly identical: 267 test cases at 33.3% passing with context, versus 233 test cases at 32.2% without. What context actually does in Python is push gpt-4o-mini to write more test cases per file — most likely because it now sees more of the API surface — and the per-file, all-tests-must-pass criterion penalises those extra tests by failing the whole file if any one of them trips over an unrelated behavioural detail. We traced one such case, `flask.logging.has_level_handler`: an identical misunderstanding of logger-hierarchy propagation appears in both arms, unrelated to the presence or absence of context. This is a second-order validity threat worth flagging for future work: an all-or-nothing file-level criterion penalises suite thoroughness by turning one behavioural assertion error into an automatic file-level failure, so arms with nearly identical per-test correctness can look dramatically different depending on how many extra tests the model wrote. It is unlikely to affect future iterations that move to per-test metrics, but it is worth noting as a source of noise.

Implication. Static API context is mostly beneficial: it improves Java's wrong-target problem with no measurable downside, and Python's per-test performance is essentially unaffected once measured fairly. Neither result changes the RQ1–RQ3 conclusions (Section IV): with or without context, gpt-4o-mini's one-shot tests fall well short of the 80%/60% thresholds. Context costs little (no model calls beyond the initial one-shot budget) and, for Java, is a real improvement worth adding to a test-generation pipeline — but including or excluding it does not change the overall conclusion that one-shot generation is inadequate to the task.

G. RQ5 (exploratory, post-hoc): one feedback/repair iteration on top of RQ4

We committed to a single repair iteration before attempting it, and iterated exactly once: repeating a repair technique until some threshold is crossed is precisely the adaptation our protocol forbids. Every RQ4 suite that was not yet effective was sent back to gpt-4o-mini with a real failure signal and an explicit request to fix one failing aspect of the suite. Java received the 0%-coverage signal plus the target API skeleton (Maven/JVM error messages proved too noisy to use as diagnostics); Python received the fresh `pytest` failure output. Suites already effective at RQ4 (11/60 Java, and Python's per-test baseline) were carried forward unchanged rather than regenerated.

Java: further, if inconclusive, improvement. The effective rate rises again, from 11/60 (18.3%, RQ4) to 15/60 (25.0%). Among the functions that changed, 4 improved and none regressed: repair is at least as good as context across the board. The one-sided Wilcoxon test barely misses significance again at $p = 0.063$, but with an improved rank-biserial effect size of $r = 1.00$ (every non-tied pair favours repair) and a steady climb across all three stages (8.3% $\rightarrow$ 18.3% $\rightarrow$ 25.0%). Mutation score stays at a median of 0% ($n = 45$ paired functions), the same as at RQ4. Repair helps gpt-4o-mini reach the correct lines more often; just as with context, the assertions are still not strong enough to kill mutants.

Python: an actual decrease in performance, this time with no artefact to blame. The per-test-case pass rate falls to 15.5% (106/682 tests), below both RQ4's 33.3% and the first context-less run's 32.2%. File count and suite length are no longer confounders, as the RQ4-vs-original test-case numbers show; unlike the context-vs-no-context comparison, they do not explain this drop. Suite size does grow again — to 682 tests across 60 files, up from 267 at RQ4 — but gpt-4o-mini appears to react to a visible failure signal by expanding the suite rather than pinpointing the one faulty assertion and repairing it. The additional tests fail at a higher rate than the initial ones succeeded, but this pattern has no obvious mechanism or cause. We have no reason to believe it is a universal property of gpt-4o-mini, of single-shot repair, or of the failure signal itself, but the one-attempt limitation rules out further investigation.

Overall assessment of RQ4+RQ5. Two low-cost, non-fine-tuning interventions leave Java with a real, if inconclusive, increase in coverage effectiveness and no downside anywhere, and Python with no benefit from context plus an outright decrease from single-shot repair. Neither intervention closes the gap to the 80%/60% thresholds, and their uneven effects across the two languages suggest that "add more scaffolding" is not a reliable way to improve LLM test generation in general. We pre-committed to stop searching for such methods once two of them failed to meet the thresholds, and we do so here rather than attempt a third.

VI. Threats to Validity

A. Construct Validity

Our measurement procedures reported false successes three times during the experiment, before the pipeline was hardened. First, mutation scores of 100% turned up for suites that had *failed* on the unmutated code: every mutant was trivially "killed" because the suite was already broken. We made a green-on-original check mandatory before any mutation analysis, so a suite that fails on the baseline source is scored INVALID outright. Second, EvoSuite exited with code 0 (success) even when it generated no tests at all; a run of 12 functions was logged as a complete success with zero test files on disk, so the pipeline now checks for the generated file directly rather than trusting the exit code. Third, JaCoCo silently reported 0% coverage for EvoSuite tests, because EvoSuite's separate instrumenting classloader changes class identity underneath it; disabling `separateClassLoader` during measurement fixed the attribution.

One further construct threat: the two Python arms use different scoring instruments — `mutmut` for the `gpt-4o-mini` suites and our own AST parser for the Pynguin suites (Section III). Both enforce the same green-on-original gate and score over the same function ranges, but they define different operator sets, so the Python RQ2-B comparison really compares scores from two instruments. Fortunately Pynguin's median coverage on this dataset was 0.0%, so the direction of that comparison does not hinge on the instrument choice. This dataset also illustrates why coverage alone is a poor quality metric: one baseline suite reached 60% branch coverage on a function while killing none of its mutants — which is exactly why we report coverage and mutation score as a pair.

B. Internal Validity

The pilot run existed to expose integration surprises before the main measurements, and it was only partly comfortable reading. The initial Java dataset held functions whose file contents did not match the method named in the metadata — `JA-002`'s file held a `concat` method, not the `getOptionValues` the LLM was asked to test, so it reasoned about the wrong method. Every pilot LLM test was discarded and the dataset was corrected before the main run. On the Python side, 10/12 pilot measurements were invalid because their functions had been extracted without their parent modules, an issue unrelated to the LLM; the later measurements run against whole modules at pinned commits. Baseline generation had its own defects: EvoSuite's initial 60s search budget against

Randoop's 10s was uncovered, and Randoop's output files were overwritten between runs; both were fixed before the main measurements, which now use consistent budgets and per-function output filenames. Several environment and tool mismatches also had to be resolved. EvoSuite 1.2.0 needs Java 8 for the `csv`/`gson`/`joda` classes while most code was tested against JDK 11, so the EvoSuite suites carry a mix of JDK 11 (39 functions) and JDK 8 (21 functions) provenance. Multi-release JARs carried Java 21 bytecode that older ASM refused to read (`Unsupported class file major version 65`), and `commons-math` disabled JaCoCo in its own `pom.xml` via `jacoco.skip=true`. Notably, four of the six measurement-pipeline failures reported exit code 0 — only the absence of artefacts on disk revealed the problem.

C. External Validity

All results are reported for the lightweight `gpt-4o-mini-2024-07-18` only and should not be extrapolated beyond it (amendment v1.1). The dataset covers two worlds: Apache-style Java libraries and Python web libraries (`flask`, `requests`). The Python side leans heavily on `flask`: 48 of the 60 Python functions come from that package. Functions with heavy environmental requirements (GUI or network access) are under-represented, and coverage varies sharply between repositories — often near-maximal for numeric utility code, near-minimal for parser- and formatter-style classes — so RQ1's 80% threshold should be read as repository-relative rather than a universal bar.

D. Conclusion Validity

With $N = 120$ functions (60 per language), we report all measurements with non-parametric tests where applicable (Wilcoxon, Spearman), at $\alpha = 0.05$, and with effect sizes alongside every p-value. The mutation and coverage thresholds were fixed in the approved protocol before any full run. Every amendment to the pipeline (model settings, dataset, prompting method, measurement environment) is documented before the measurements it affects, which is what guards against HARKing. Mutation scores were unavailable (PIT generated no mutants) for 3/60 of the baseline cells: `JA-017` (`jfreechart`), `JA-048`, and `JA-058` (`jsoup`). These are recorded in `metrics_full.csv` with the mutation-score column empty, since those functions had no mutants to kill; the coverage figures are unaffected, as they are computed independently and before the PIT step. The same three functions also appear among the LLM's own INVALID/no-touch Java results (Section IV), for an unrelated reason — wrong-target invocation. PIT's missing mutants and the LLM's wrong-target problem were diagnosed independently; the overlap is incidental.

A separate environment issue affected only the Pynguin baseline (Python). Under Python 3.14 — the interpreter used everywhere else in the measurement pipeline — Pynguin's bytecode instrumentation failed outright, generating empty test suites (0% coverage) for every module. Verbose logging traced it to a Pynguin internals problem: its master–worker process communication broke under Python 3.14, a genuine interpreter-level incompatibility. The fix was a dedicated Python 3.10 virtual environment for Pynguin invocations only — the version most Pynguin releases are validated against. This adjustment touches none of the pre-registered metrics, thresholds, datasets, or models, and was applied uniformly across all Pynguin measurements before any data were collected, which is why it is not an amendment. Even with the fix, Pynguin's median coverage on the Python dataset stayed at 0.0% (Section IV).

A late audit of that 0.0% figure shows it blends two distinct causes, and we report the distinction rather than leave it implicit. For the largest modules Pynguin produced no output at all within its 90-second budget — `flask/app.py` (1625 LOC) and `flask/cli.py` (1127 LOC) among them — and that failure tracks module size rather than function complexity: every module of 385 LOC or fewer yielded a measurable suite, every module of 682 LOC or more yielded none. Since our sampling unit is the function while Pynguin operates on whole modules, functions drawn from very large files handed the baseline a task it could not finish in budget; scoring those as 0% is defensible under the pre-registered rule, but it is a statement about applicability, not about the quality of the tests Pynguin writes. A second group is more troubling: there, Pynguin did produce a working suite — it self-reports 40% coverage on `requests.adapters` — but our green-on-original check runs against the whole module suite, so one failing test zeroed every function in that module. Restricted to the 12 functions whose module cleared the gate, Pynguin's median branch coverage is 40.0%, not 0.0%. This is the same all-or-nothing artefact we describe for the Python LLM arm in Section V-F, and we had not applied that insight to the baseline. It does not change the RQ1–RQ3 conclusions, which concern `gpt-4o-mini` rather than Pynguin, but it does mean the RQ2-B Python comparison should be read as gpt-4o-mini against a baseline that our own harness partly suppressed. Re-measuring Pynguin with a per-function gate is the first item of our errata.

VII. Conclusion

This paper evaluates the one-shot unit-test generation capability of GPT-4o-mini on 120 real-world Java and Python functions of moderate cyclomatic complexity (CC 5–10), assessed inside their host projects against EvoSuite and Randoop (60s Java baselines) and Pynguin (90s Python baseline), under three research questions. The main contribution is an empirical assessment of how useful one-shot test generation actually is in this setting, together with a set of directions for future work on the task.

RQ1/RQ2. GPT-4o-mini did not pass the 80% coverage threshold at any level of analysis. On the pooled effective subset (the 28 functions whose suites compiled and invoked the target), median branch coverage was 75.0% ($p = 0.84$, $r = -0.22$). For context, the same 60 Java functions reached a median branch coverage of 55.0% under EvoSuite and 21.1%

under Randoop — so the 80% bar was demanding for every tool, not just the LLM. Mutation scores were weaker still: among the suites with a defined score, the median was 36.84% for Python ($n = 17$) and 0% for Java ($n = 54$), none clearing the 60% threshold. Paired Wilcoxon tests found both Java baselines significantly stronger than GPT-4o-mini (EvoSuite $p < 0.001$, Randoop $p = 0.003$), each with an effect size of $r = -1.00$ — the baseline won every Java function where the two suites differed. For Python the direction reversed ($p = 0.010$, $r = +0.85$), but Pynguin achieved 0.0% median coverage on this sample (Section VI), so that result says only that GPT-4o-mini can beat a tool that failed the task, not that its own Python tests are strong.

RQ3. The relationship between CC and coverage was weak across the sampled range ($\rho = +0.18$, $p = 0.41$ Python; $\rho = +0.13$, $p = 0.83$ Java; $\rho = +0.16$, $p = 0.40$ pooled). Inside the CC 5–10 band, test-generation effectiveness came down to whether the suite compiled and invoked the target (Section IV), not to a graded decline with complexity. Locating a true "breaking point" would require a larger study spanning below CC 5 and above CC 10.

Two lessons stand out. First, evaluating LLM-generated tests on functions taken out of context systematically understates the difficulty of real-world code: functions from mature projects lean on their surrounding module — their imports, class hierarchy, and name resolution — and an evaluation that ignores that context flatters the model. Second, the measurement pipeline deserves the same scrutiny as the model: most of the failures we hit were silent (exit code 0, no artefacts) and had to be caught by inspecting the report files on disk (Section VI). Coverage alone is likewise too coarse a signal — without the paired mutation analysis, several of the Python suites in Section IV would have looked adequate when they were not.

RQ4 and RQ5 each describe a post-hoc intervention — real API context, then a single feedback/repair iteration — aimed at the shortcomings of the plain one-shot setup. Neither closed the gap. For Java, the effective-suite rate climbed across the three stages (no intervention $\rightarrow$ context $\rightarrow$ one repair iteration): 8.3% $\rightarrow$ 18.3% $\rightarrow$ 25.0% ($p = 0.074$ then $p = 0.063$, $n = 60$), with mutation score unchanged at 0% throughout. For Python, neither the added context nor the single repair iteration improved matters: context was neutral once measured per test, and the repair step produced an outright per-test regression, most likely because the model answered failures by padding the suite with more tests rather than fixing the failing assertions. Consistent with our anti-HARKing commitment, the exploratory arm stopped at these two interventions, with no further attempt to find a threshold-crossing method.

Five directions for future work follow. First, expand the Java sample to test whether the RQ4/RQ5 coverage trend reaches statistical significance. Second, attempt an analogous iterative repair for Python but with an explicit stopping criterion (e.g. a change in suite size below some fixed percentage between iterations), to separate first-attempt gains from the effect of repeated retries. Third, replicate the analysis with larger LLMs (GPT-4o or otherwise) to separate the effect of model capacity from prompt engineering. Fourth, broaden the Python sample to further libraries and take both languages to higher CC bands to observe trends beyond CC 10. Finally, automate the evaluation itself for in-situ testing inside a CI/CD pipeline, so interventions like RQ4's can run as self-improving processes. Each of these would benefit from pre-registration.

Overall, our findings do not support one-shot test generation as a viable substitute for existing tools on real-world code. Closing that gap will take feedback loops, retrieval mechanisms, and repair strategies to bring one-shot generation up towards the effectiveness of dedicated test-generation tools.

AI Use Disclosure

This document was originally drafted by human authors. Automated evaluation and reporting sections, including test generation and analysis, were produced and refined using AI-based large language models.

References

[1] W. Wang, C. Yang, Z. Wang, Y. Huang, Z. Chu, D. Song, L. Zhang, A. R. Chen, and L. Ma, "TestEval: Benchmarking large language models for test case generation," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 3547–3562.

[2] A. Kumar *et al.*, "Empirical evaluation of advanced LLMs on automated unit test generation for Java applications," *IEEE Access*, vol. 13, pp. 1245–1260, 2025.

[3] Lira *et al.*, "Evaluating the effectiveness and cost-efficiency of large language models in automated unit test generation," in *Anais do Simpósio Brasileiro de Qualidade de Software (SBQS)*, 2025. [Online]. Available: https://sol.sbc.org.br/index.php/sbqs/article/view/39000

[4] A. Chudic and G. Çalıklı, "Automated test suite enhancement using large language models with few-shot prompting," 2026.

[5] K. Jasti, T. P. Sahu, R. Pentyala, M. Mohit, and V. Yelleti, "FeedbackLLM: Metadata-driven multi-agentic language-agnostic test case generator with evolving prompt and coverage feedback," 2026.

[6] M. Haratian *et al.*, "PR-aware test case generation: An empirical comparison of search-based and LLM-based approaches," 2026.

[7] G. Sapozhnikov *et al.*, "An empirical study on the structural coverage and compilation rate of ChatGPT-generated unit tests," in *Proc. ICSE Companion*, 2024.

[8] Q. Zhang, Y. Shang, C. Fang, S. Gu, J. Zhou, and Z. Chen, "TestBench: Evaluating class-level test case generation capability of large language models," 2024.

[9] D. Konstantinou *et al.*, "Evaluating plain-LLM test generation across diverse Java projects: A multi-model analysis," 2026.

[10] R. Jain and C. Le Goues, "TestForge: Feedback-driven, agentic test suite generation," 2025.

[11] G. Fraser and A. Arcuri, "EvoSuite: Automatic test suite generation for object-oriented software," in *Proc. ESEC/FSE*, 2011.

[12] C. Pacheco and M. D. Ernst, "Randoop: Feedback-directed random testing for Java," in *Proc. OOPSLA Companion*, 2007.

[13] S. Lukasczyk and G. Fraser, "Pynguin: Automated unit test generation for Python," in *Proc. ICSE Companion*, 2022.

[14] W. C. Ouédraogo *et al.*, "Prompt engineering in LLMs for automated unit test generation: A large-scale study," *Empirical Software Engineering*, 2025.

[15] S. Gu *et al.*, "Improving LLM-based unit test generation via template-based repair," 2025, aCM Trans. Softw. Eng. Methodol. (TOSEM).

[16] L. Broide, R. Stern, and A. Mordoch, "EvoGPT: Leveraging LLM-driven seed diversity to improve search-based test suite generation," 2026.

[17] Zeng *et al.*, "A logic-guided and explainable approach to LLM-based unit test generation," *Applied Sciences*, vol. 16, no. 5, p. 2542, 2026.

[18] M. Konstantinou, R. Degiovanni, J. M. Zhang, M. Harman, and M. Papadakis, "YATE: The role of test repair in LLM-based unit test generation," 2025.

[19] P. Xue, Y. Zhang, Z. Yang, X. Ren, X. Li, P. Hu, L. Wu, and K. Li, "DISTINCT: A description-guided branch-consistency analysis framework for non-regressive test case generation," 2025, iEEE Trans. Softw. Eng. (TSE).
[20] R. Liu *et al.*, "Type-aware LLM-based regression test generation for Python programs," 2025.
[21] G. Wang, C. Yang, X. Zhou, Z. Sun, B. Wang, D. Lo, and D. Hao, "TestDecision: Sequential test suite generation via greedy optimization and reinforcement learning," 2026.
[22] A. Moradi Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais, "Effective test generation using pre-trained large language models and mutation testing," 2023.
[23] G. Antal *et al.*, "Leveraging GPT-4 for vulnerability-witnessing unit test generation," in *Proc. EASE*, 2025.
[24] Silva *et al.*, "LLMs as test generators: A comparative benchmarking study," in *Anais do Simpósio Brasileiro de Engenharia de Software (SBES)*, 2025. [Online]. Available: https://sol.sbc.org.br/index.php/sbes/article/view/36983
[25] H. Coles, T. Laurent, C. Henard, M. Papadakis, and A. Ventresque, "PIT: A practical mutation testing tool for Java," in *Proc. ISSTA*, 2016.